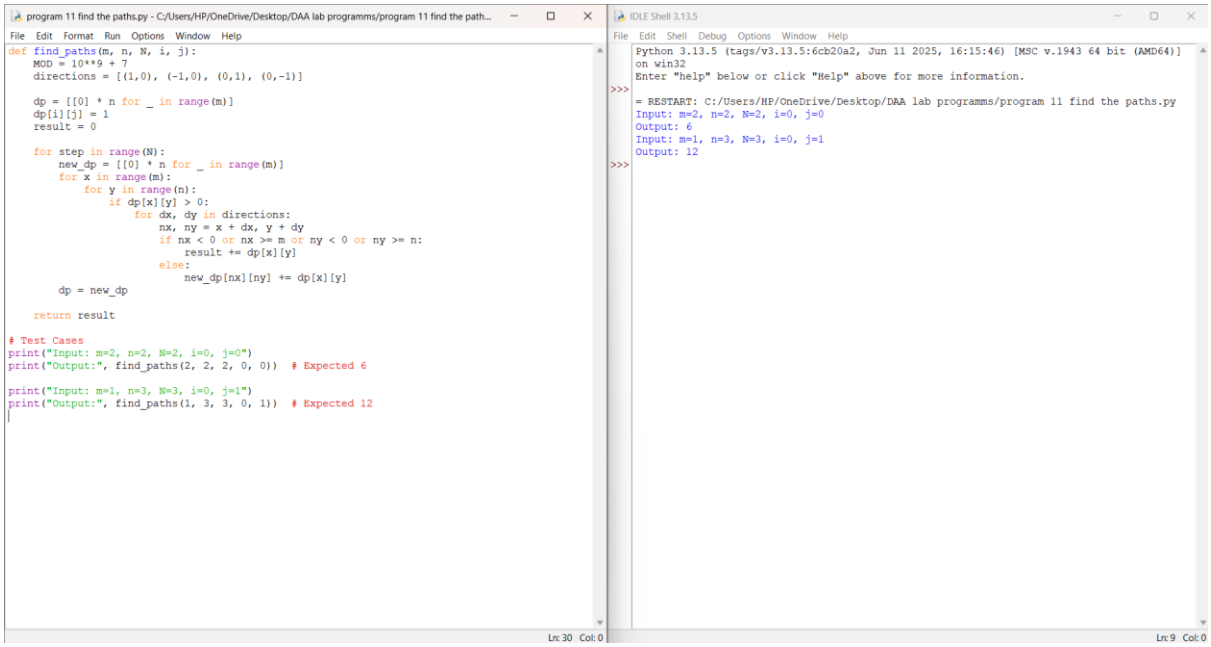
DESIGN AND ANALYSIS OF ALGORITHM

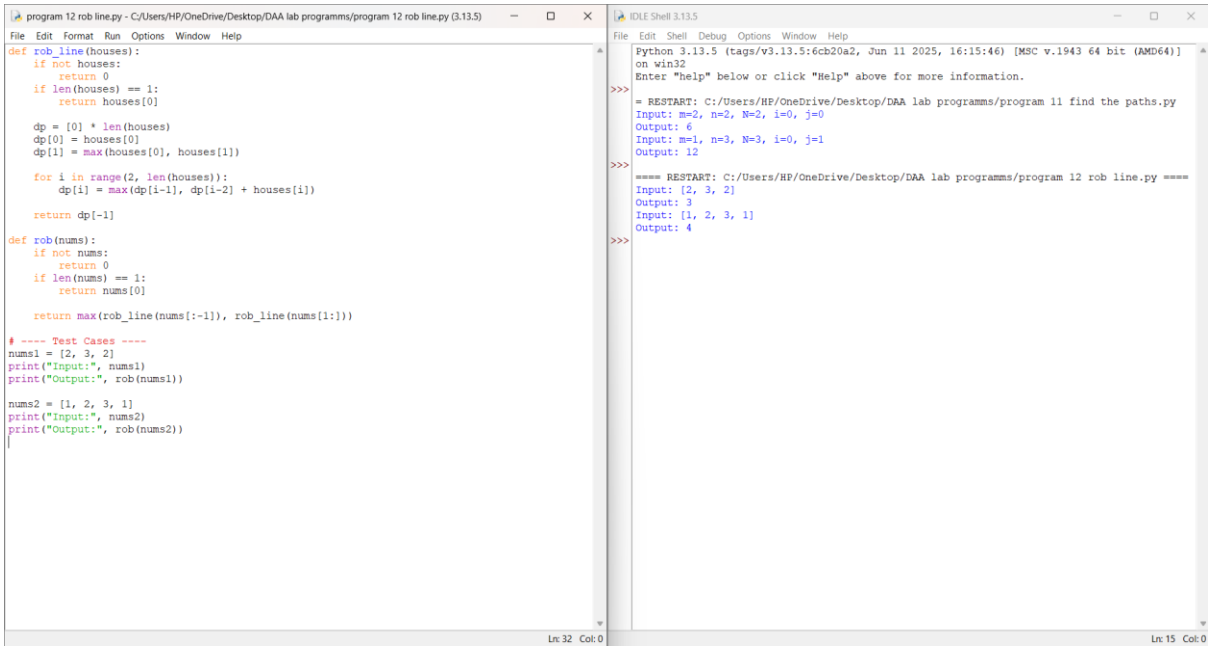
1.



```
def find_paths(m, n, N, i, j):
    MOD = 10**9 + 7
    directions = [(1,0), (-1,0), (0,1), (0,-1)]

    dp = [[0] * n for _ in range(m)]
    dp[i][j] = 1
    result = 0

    for step in range(N):
        new_dp = [[0] * n for _ in range(m)]
        for x in range(m):
            for y in range(n):
                if dp[x][y] > 0:
                    for dx, dy in directions:
                        nx, ny = x + dx, y + dy
                        if nx < 0 or nx >= m or ny < 0 or ny >= n:
                            result += dp[x][y]
                        else:
                            new_dp[nx][ny] += dp[x][y]
        dp = new_dp

    return result

# Test Cases
print("Input: m=2, n=2, N=2, i=0, j=0")
print("Output:", find_paths(2, 2, 2, 0, 0))  # Expected 6

print("Input: m=1, n=3, N=3, i=0, j=1")
print("Output:", find_paths(1, 3, 3, 0, 1))  # Expected 12
```

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 11 find the paths.py
Input: m=2, n=2, N=2, i=0, j=0
Output: 6
Input: m=1, n=3, N=3, i=0, j=1
Output: 12
>>>
```

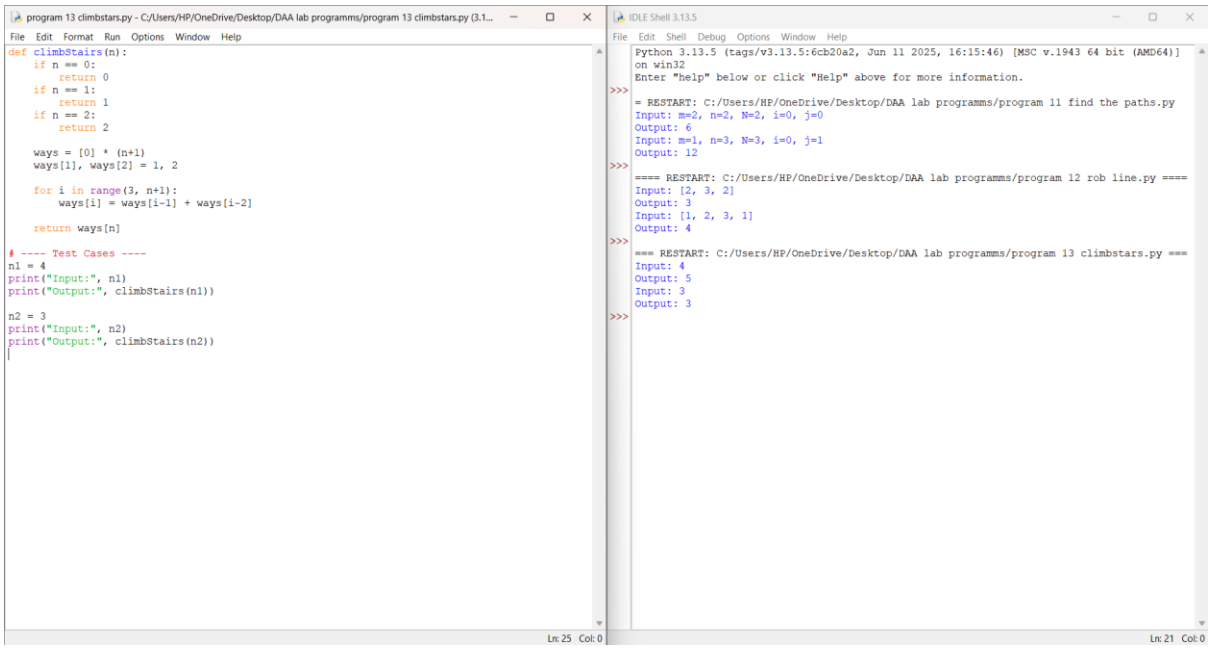
2.



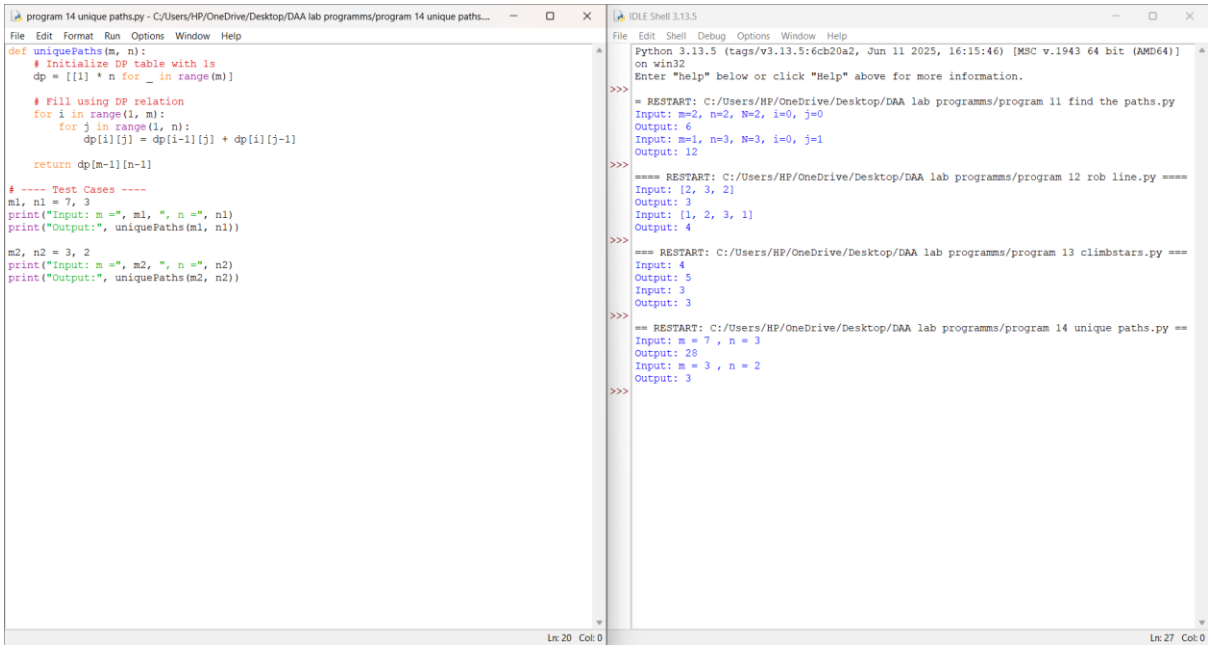
```
def rob_line(houses):
    if not houses:
        return 0
    if len(houses) == 1:
        return houses[0]

    dp = [0] * len(houses)
    dp[0] = houses[0]
    dp[1] = max(houses[0], houses[1])

    for i in range(2, len(houses)):
        dp[i] = max(dp[i-1], dp[i-2] + houses[i])

    return dp[-1]

def rob(nums):
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]

    return max(rob_line(nums[:-1]), rob_line(nums[1:]))

# ---- Test Cases ----
nums1 = [2, 3, 2]
print("Input:", nums1)
print("Output:", rob(nums1))

nums2 = [1, 2, 3, 1]
print("Input:", nums2)
print("Output:", rob(nums2))
```

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 11 find the paths.py
Input: m=2, n=2, N=2, i=0, j=0
Output: 6
Input: m=1, n=3, N=3, i=0, j=1
Output: 12
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 12 rob line.py ====
Input: [2, 3, 2]
Output: 3
Input: [1, 2, 3, 1]
Output: 4
>>>
```

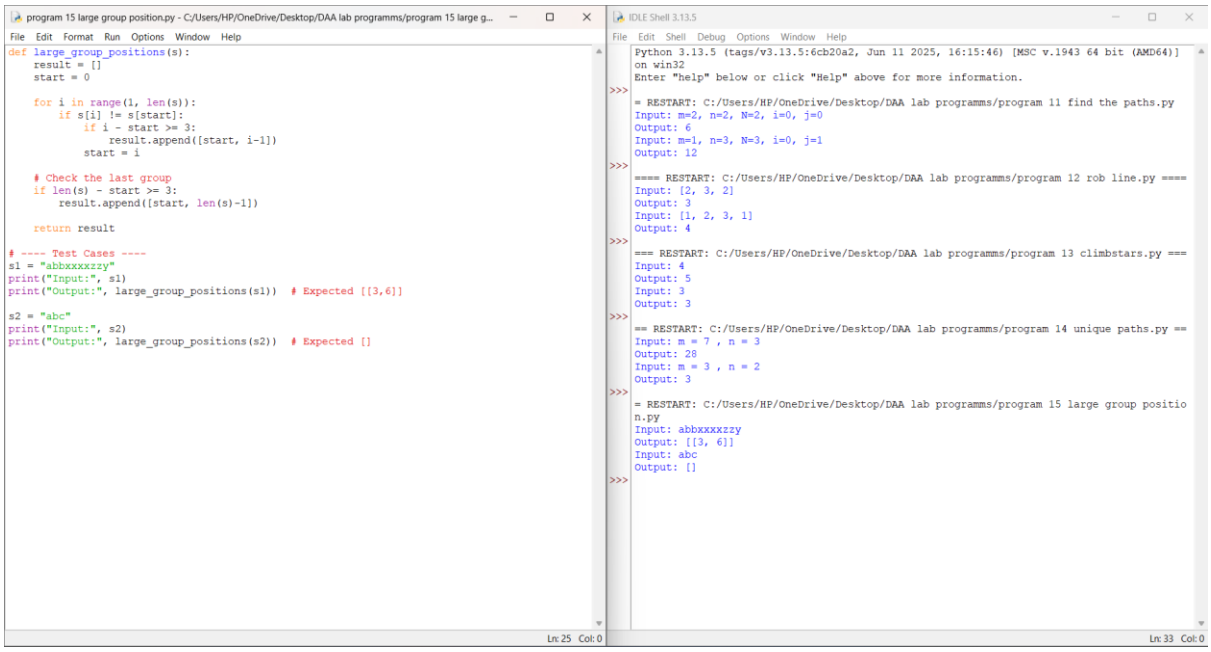
3.


program 13 climbstars.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 13 climbstars.py (3.1...
File Edit Format Run Options Window Help
def climbStairs(n):
if n == 0:
return 0
if n == 1:
return 1
if n == 2:
return 2
ways = [0] * (n+1)
ways[1], ways[2] = 1, 2
for i in range(3, n+1):
ways[i] = ways[i-1] + ways[i-2]
return ways[n]
# ---- Test Cases ----
n1 = 4
print("Input:", n1)
print("Output:", climbStairs(n1))
n2 = 3
print("Input:", n2)
print("Output:", climbStairs(n2))
Ln: 25 Col: 0
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 11 find the paths.py
Input: m=2, n=2, N=2, i=0, j=0
Output: 6
Input: m=1, n=3, N=3, i=0, j=1
Output: 12
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 12 rob line.py ====
Input: [2, 3, 2]
Output: 3
Input: [1, 2, 3, 1]
Output: 4
>>>
=== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 13 climbstars.py ===
Input: 4
Output: 5
Input: 3
Output: 3
>>>
Ln: 21 Col: 0

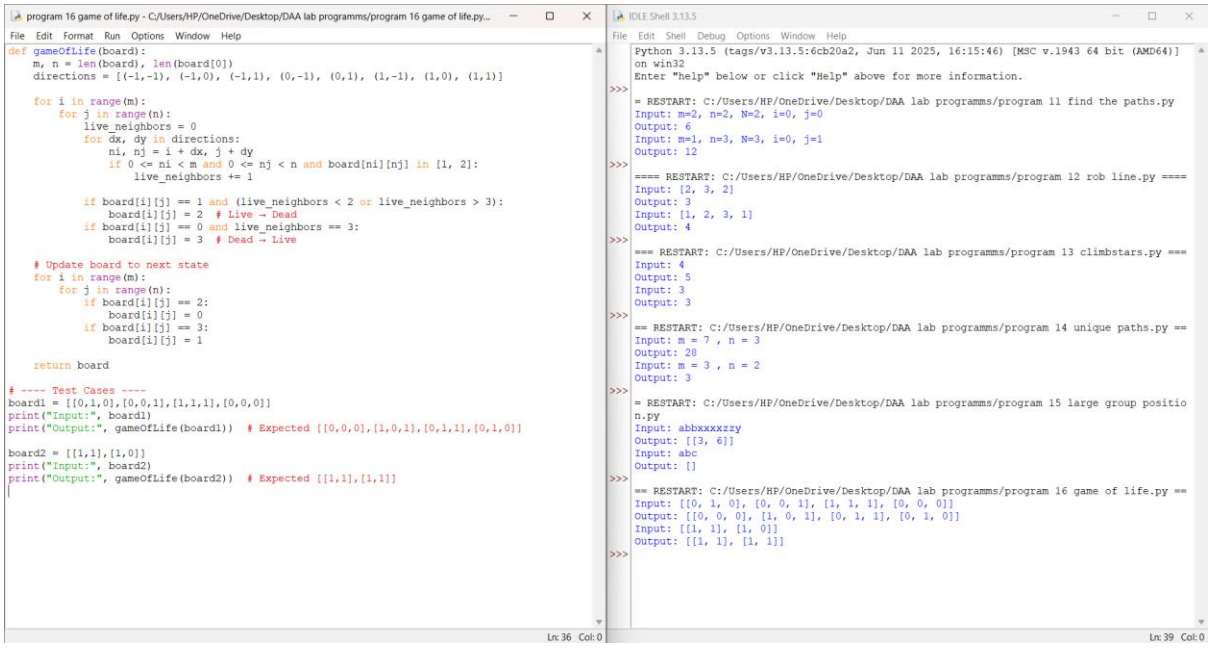
4.


program 14 unique paths.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 14 unique paths....
File Edit Format Run Options Window Help
def uniquePaths(m, n):
# Initialize DP table with 1s
dp = [[1] * n for _ in range(m)]
# Fill using DP relation
for i in range(1, m):
for j in range(1, n):
dp[i][j] = dp[i-1][j] + dp[i][j-1]
return dp[m-1][n-1]
---- Test Cases ----
m1, n1 = 7, 3
print("Input: m =", m1, ", n =", n1)
print("Output:", uniquePaths(m1, n1))
m2, n2 = 3, 2
print("Input: m =", m2, ", n =", n2)
print("Output:", uniquePaths(m2, n2))
Ln: 20 Col: 0
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 11 find the paths.py
Input: m=2, n=2, N=2, i=0, j=0
Output: 6
Input: m=1, n=3, N=3, i=0, j=1
Output: 12
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 12 rob line.py ====
Input: [2, 3, 2]
Output: 3
Input: [1, 2, 3, 1]
Output: 4
>>>
=== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 13 climbstars.py ===
Input: 4
Output: 5
Input: 3
Output: 3
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 14 unique paths.py ==
Input: m = 7 , n = 3
Output: 28
Input: m = 3 , n = 2
Output: 3
>>>
Ln: 27 Col: 0

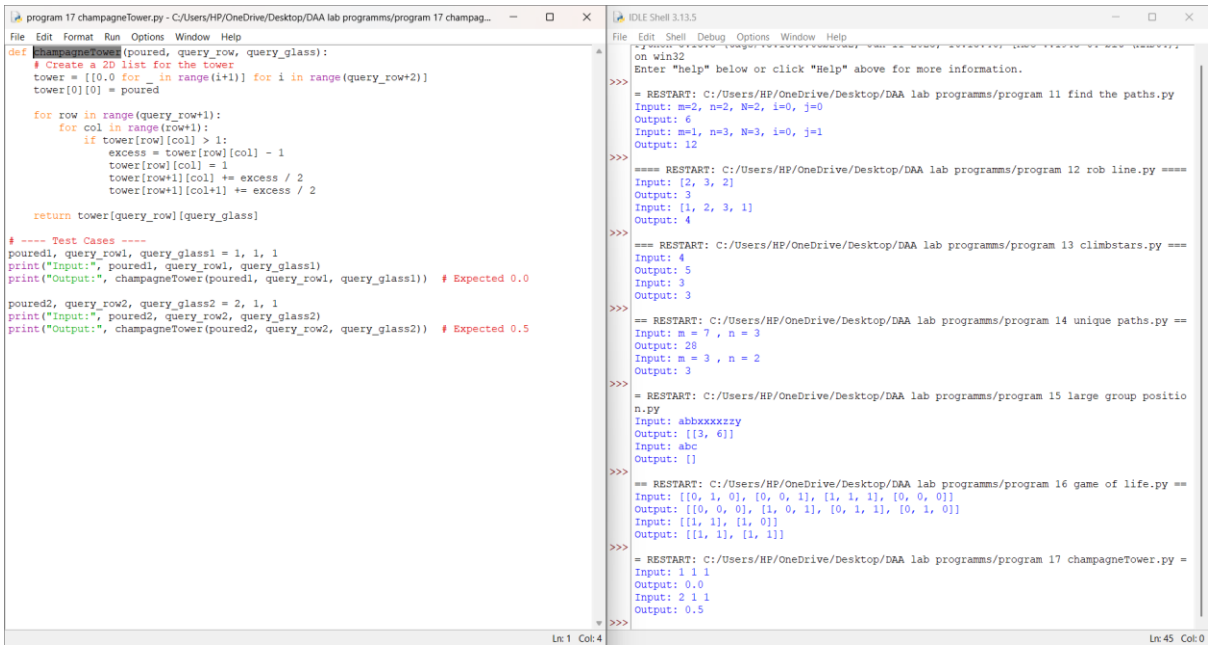
5.


program 15 large group position.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 15 large g...
File Edit Format Run Options Window Help
def large_group_positions(s):
result = []
start = 0
for i in range(1, len(s)):
if s[i] != s[start]:
if i - start >= 3:
result.append([start, i-1])
start = i
# Check the last group
if len(s) - start >= 3:
result.append([start, len(s)-1])
return result
# ---- Test Cases ----
s1 = "abbxxxxzzy"
print("Input:", s1)
print("Output:", large_group_positions(s1)) # Expected [[3,6]]
s2 = "abc"
print("Input:", s2)
print("Output:", large_group_positions(s2)) # Expected []
Ln: 25 Col: 0
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 11 find the paths.py
Input: m=2, n=2, N=2, i=0, j=0
Output: 6
Input: m=1, n=3, N=3, i=0, j=1
Output: 12
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 12 rob line.py ====
Input: [2, 3, 2]
Output: 3
Input: [1, 2, 3, 1]
Output: 4
>>>
=== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 13 climbstars.py ===
Input: 4
Output: 5
Input: 3
Output: 3
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 14 unique paths.py ==
Input: m = 7 , n = 3
Output: 28
Input: m = 3 , n = 2
Output: 3
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 15 large group positio
n.py
Input: abbxxxxzzy
Output: [[3, 6]]
Input: abc
Output: []
>>>
Ln: 33 Col: 0

6.


program 16 game of life.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 16 game of life.py...
File Edit Format Run Options Window Help
def gameOfLife(board):
m, n = len(board), len(board[0])
directions = [(-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1)]
for i in range(m):
for j in range(n):
live_neighbors = 0
for dx, dy in directions:
ni, nj = i + dx, j + dy
if 0 <= ni < m and 0 <= nj < n and board[ni][nj] in [1, 2]:
live_neighbors += 1
if board[i][j] == 1 and (live_neighbors < 2 or live_neighbors > 3):
board[i][j] = 2 # Live → Dead
if board[i][j] == 0 and live_neighbors == 3:
board[i][j] = 3 # Dead → Live
# Update board to next state
for i in range(m):
for j in range(n):
if board[i][j] == 2:
board[i][j] = 0
if board[i][j] == 3:
board[i][j] = 1
return board
# ---- Test Cases ----
board1 = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
print("Input:", board1)
print("Output:", gameOfLife(board1)) # Expected [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]
board2 = [[1,1],[1,0]]
print("Input:", board2)
print("Output:", gameOfLife(board2)) # Expected [[1,1],[1,1]]
Ln: 36 Col: 0
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 11 find the paths.py
Input: m=2, n=2, N=2, i=0, j=0
Output: 6
Input: m=1, n=3, N=3, i=0, j=1
Output: 12
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 12 rob line.py ====
Input: [2, 3, 2]
Output: 3
Input: [1, 2, 3, 1]
Output: 4
>>>
=== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 13 climbstars.py ===
Input: 4
Output: 5
Input: 3
Output: 3
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 14 unique paths.py ==
Input: m = 7 , n = 3
Output: 28
Input: m = 3 , n = 2
Output: 3
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 15 large group positio
n.py
Input: abbxxxxzzy
Output: [[3, 6]]
Input: abc
Output: []
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 16 game of life.py ==
Input: [[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]]
Output: [[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
Input: [[1, 1], [1, 0]]
Output: [[1, 1], [1, 1]]
>>>
Ln: 39 Col: 0

7.



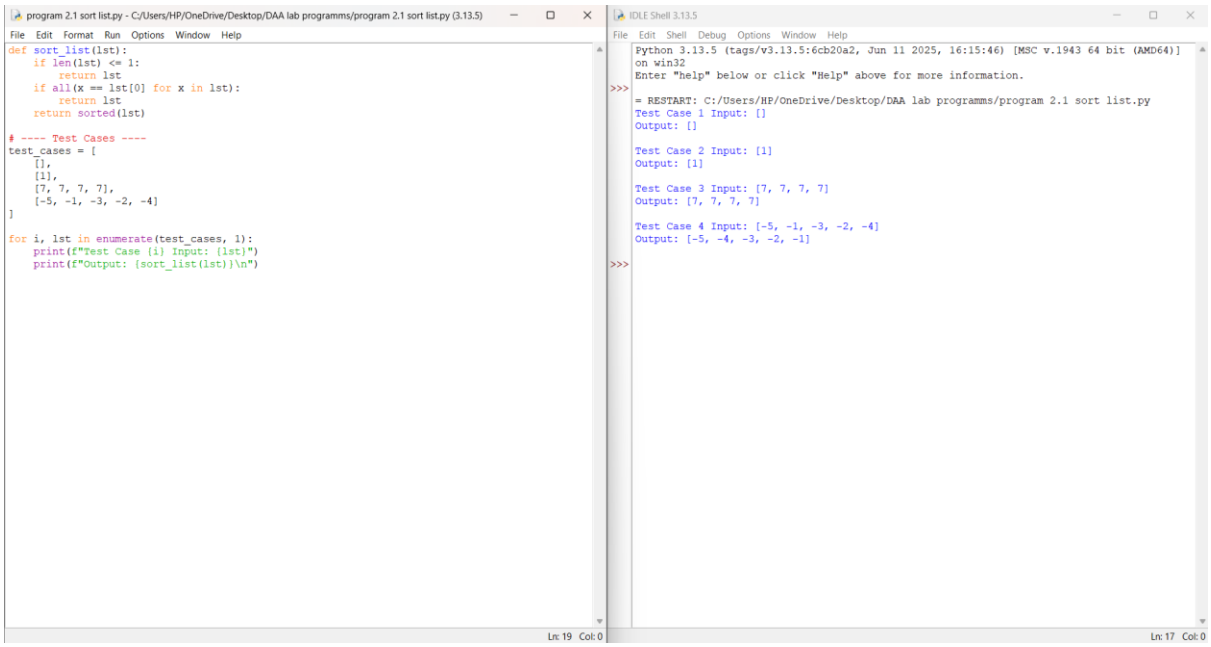
```
program 17 champagneTower.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 17 champag...
File  Edit  Format  Run  Options  Window  Help
def champagneTower(poured, query_row, query_glass):
    # Create a 2D list for the tower
    tower = [[0.0 for _ in range(i+1)] for i in range(query_row+2)]
    tower[0][0] = poured

    for row in range(query_row+1):
        for col in range(row+1):
            if tower[row][col] > 1:
                excess = tower[row][col] - 1
                tower[row][col] = 1
                tower[row+1][col] += excess / 2
                tower[row+1][col+1] += excess / 2

    return tower[query_row][query_glass]

# ---- Test Cases ----
poured1, query_row1, query_glass1 = 1, 1, 1
print("Input:", poured1, query_row1, query_glass1)
print("Output:", champagneTower(poured1, query_row1, query_glass1))  # Expected 0.0

poured2, query_row2, query_glass2 = 2, 1, 1
print("Input:", poured2, query_row2, query_glass2)
print("Output:", champagneTower(poured2, query_row2, query_glass2))  # Expected 0.5
```

Ln: 1 Col: 4

```
IDLE Shell 3.13.5
File  Edit  Shell  Debug  Options  Window  Help
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 11 find the paths.py
Input: m=2, n=2, N=2, i=0, j=0
Output: 6
Input: m=1, n=3, N=3, i=0, j=1
Output: 12
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 12 rob line.py ====
Input: [2, 3, 2]
Output: 3
Input: [1, 2, 3, 1]
Output: 4
>>>
=== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 13 climbstars.py ===
Input: 4
Output: 5
Input: 3
Output: 3
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 14 unique paths.py ==
Input: m = 7 , n = 3
Output: 28
Input: m = 3 , n = 2
Output: 3
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 15 large group positio
n.py
Input: abbxxxxzzy
Output: [[3, 6]]
Input: abc
Output: []
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 16 game of life.py ==
Input: [[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]]
Output: [[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
Input: [[1, 1], [1, 0]]
Output: [[1, 1], [1, 1]]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 17 champagneTower.py =
Input: 1 1 1
Output: 0.0
Input: 2 1 1
Output: 0.5
>>>
```

Ln: 45 Col: 0

8.



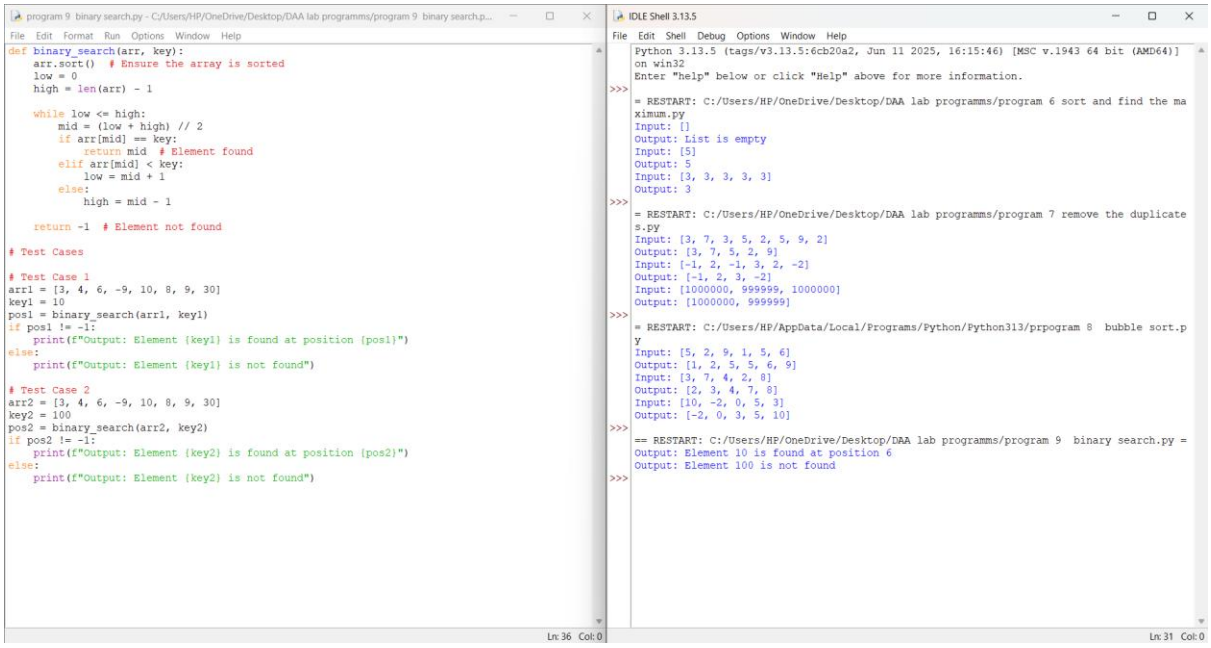
```
program 2.1 sort list.py - C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.1 sort list.py (3.13.5)
File  Edit  Format  Run  Options  Window  Help
def sort_list(lst):
    if len(lst) <= 1:
        return lst
    if all(x == lst[0] for x in lst):
        return lst
    return sorted(lst)

# ---- Test Cases ----
test_cases = [
    [],
    [1],
    [7, 7, 7, 7],
    [-5, -1, -3, -2, -4]
]

for i, lst in enumerate(test_cases, 1):
    print(f"Test Case {i} Input: {lst}")
    print(f"Output: {sort_list(lst)}\n")
```

Ln: 19 Col: 0

```
IDLE Shell 3.13.5
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.1 sort list.py
Test Case 1 Input: []
Output: []

Test Case 2 Input: [1]
Output: [1]

Test Case 3 Input: [7, 7, 7, 7]
Output: [7, 7, 7, 7]

Test Case 4 Input: [-5, -1, -3, -2, -4]
Output: [-5, -4, -3, -2, -1]

>>>
```

Ln: 17 Col: 0

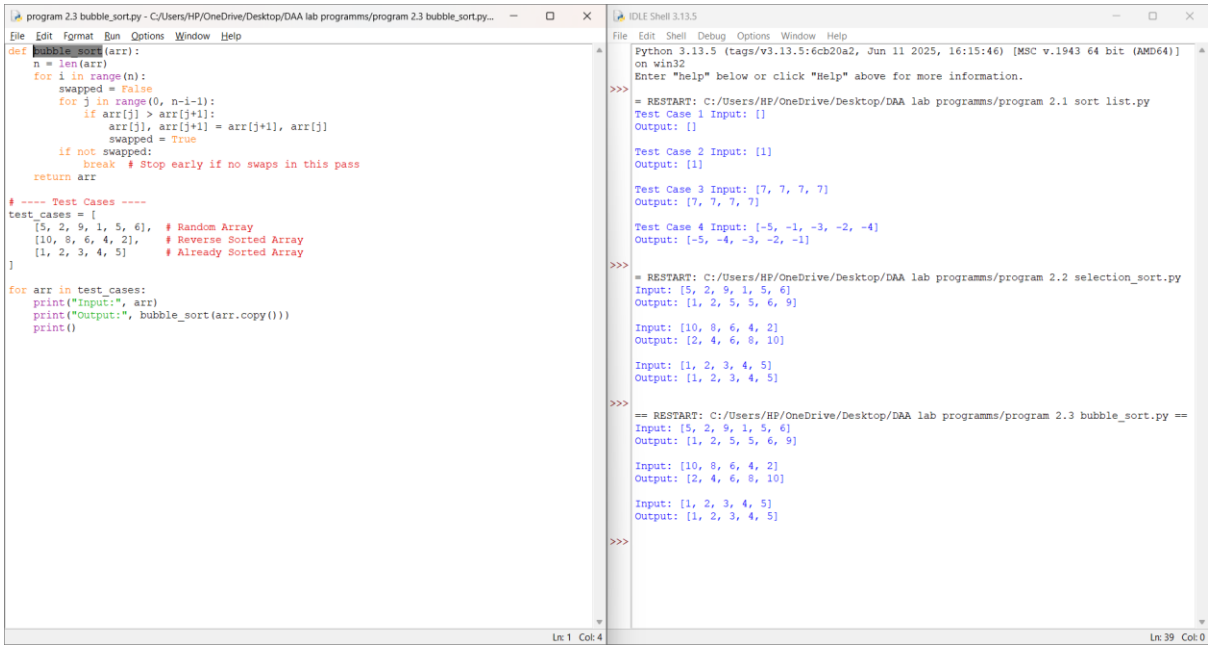
9.



```
def binary_search(arr, key):
    arr.sort()  # Ensure the array is sorted
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid  # Element found
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1  # Element not found

# Test Cases

# Test Case 1
arr1 = [3, 4, 6, -9, 10, 8, 9, 30]
key1 = 10
pos1 = binary_search(arr1, key1)
if pos1 != -1:
    print(f"Output: Element {key1} is found at position {pos1}")
else:
    print(f"Output: Element {key1} is not found")

# Test Case 2
arr2 = [3, 4, 6, -9, 10, 8, 9, 30]
key2 = 100
pos2 = binary_search(arr2, key2)
if pos2 != -1:
    print(f"Output: Element {key2} is found at position {pos2}")
else:
    print(f"Output: Element {key2} is not found")
```

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 6 sort and find the maximum.py
Input: []
Output: List is empty
Input: [5]
Output: 5
Input: [3, 3, 3, 3, 3]
Output: 3
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 7 remove the duplicates.py
Input: [3, 7, 3, 5, 2, 5, 9, 2]
Output: [3, 7, 5, 2, 9]
Input: [-1, 2, -1, 3, 2, -2]
Output: [-1, 2, 3, -2]
Input: [1000000, 999999, 1000000]
Output: [1000000, 999999]
>>>
= RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python313/prpogram 8  bubble sort.py
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
Input: [3, 7, 4, 2, 8]
Output: [2, 3, 4, 7, 8]
Input: [10, -2, 0, 5, 3]
Output: [-2, 0, 3, 5, 10]
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 9  binary search.py =
Output: Element 10 is found at position 6
Output: Element 100 is not found
>>>
```

10.



```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break  # Stop early if no swaps in this pass
    return arr

# ---- Test Cases ----
test_cases = [
    [5, 2, 9, 1, 5, 6],  # Random Array
    [10, 8, 6, 4, 2],    # Reverse Sorted Array
    [1, 2, 3, 4, 5]      # Already Sorted Array
]

for arr in test_cases:
    print("Input:", arr)
    print("Output:", bubble_sort(arr.copy()))
    print()
```

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.1 sort list.py
Test Case 1 Input: []
Output: []

Test Case 2 Input: [1]
Output: [1]

Test Case 3 Input: [7, 7, 7, 7]
Output: [7, 7, 7, 7]

Test Case 4 Input: [-5, -1, -3, -2, -4]
Output: [-5, -4, -3, -2, -1]

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.2 selection_sort.py
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]

Input: [10, 8, 6, 4, 2]
Output: [2, 4, 6, 8, 10]

Input: [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programms/program 2.3 bubble_sort.py ==
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]

Input: [10, 8, 6, 4, 2]
Output: [2, 4, 6, 8, 10]

Input: [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

>>>
```